DIAGRAMA DE CLASES UML

Proyecto Final: Scooby-Doo — El Misterio de Cartagena (Años 1600)

Autores: Daniel Mosquera & Juan Solarte — **Asignatura:** Informática II

Facultad de Ingeniería — Universidad de Antioquia (Semestre 2026-2)

1. Distribución Modular del Sistema por Paquetes Lógicos

Paquetes del Sistema y Flujo de Dependencias

1. **Paquete NucleoYTiposCompartidos (`nucleo`):** Tipos geométricos 2D, medidores de recursos reutilizables (estamina, provisiones y tiempo), generador de aleatoriedad determinista y jerarquía de excepciones propias.
2. **Paquete ModelosDeFisica (`fisica`):** Modelos cinemáticos continuos parametrizables por dificultad: péndulo amortiguado (lámpara colgante), movimiento circular uniforme (patrulla de centinela) y balística parabólica (bala de cañón).
3. **Paquete EntornoYNavegacion (`mapa`):** Representación de la cuadrícula de Cartagena, zonas públicas y restringidas, puertas secundarias bloqueables y cálculo de rutas transitables con algoritmo A*.
4. **Paquete EntidadesYPersonajes (`entidades`):** Jerarquía polimórfica que agrupa al protagonista Scooby-Doo, los 4 sospechosos, el villano encubierto, el fantasma, soldados aliados, amigos colaboradores y objetos coleccionables.
5. **Paquete AgenteAutonomoEInteligencia ():** Componente investigativo que implementa el ciclo formal de 4 etapas: Percepción sensorial, Razonamiento lógico, Acción física y Aprendizaje/Memoria acumulativa.
6. **Paquete GestionDelJuegoYNiveles (`juego`):** Orquestación de la sesión, reloj maestro con paso delta (`deltaTiempo`), perfiles de dificultad, generación procedural del caso criminal y soporte para Nivel 1 y Nivel 2.

2. Paquete 1: Núcleo, Tipos Básicos, Recursos y Excepciones

VectorDosDimensiones — *Estructura de Datos Geométrica*

Atributos Públicos:

- `+ posicionEnEjeX : double = 0.0`
- `+ posicionEnEjeY : double = 0.0`

Métodos y Operadores:

- `+ VectorDosDimensiones(posicionInicialX: double = 0.0, posicionInicialY: double = 0.0)`
- `+ operator+(otroVector: const VectorDosDimensiones&) const : VectorDosDimensiones`
- `+ operator-(otroVector: const VectorDosDimensiones&) const : VectorDosDimensiones`
- `+ operator*(factorEscalar: double) const : VectorDosDimensiones`
- `+ calcularDistanciaAlCuadrado(otroVector: const VectorDosDimensiones&) const : double`
- `+ calcularLongitudEuclidiana() const : double`
- `+ obtenerVectorNormalizado() const : VectorDosDimensiones`

MedidorDeRecursoAcumulable — *Control de Barras Continuas*

Atributos Privados:

- `- cantidadActualDelRecurso : double`
- `- capacidadMaximaPermitida : double`

Métodos Públicos:

- `+ MedidorDeRecursoAcumulable(capacidadMaxima: double, valorInicial: double)`
- `+ agregarCantidadAlRecurso(cantidadAAgregar: double) : void`
- `+ consumirCantidadDelRecurso(cantidadAConsumir: double) : bool`

- `+ rellenarAlMaximoCapacidad() : void`
- `+ estaCompletamenteVacio() const : bool`
- `+ estaCompletamenteLleno() const : bool`
- `+ obtenerPorcentajeRestante() const : double`
- `+ obtenerCantidadActual() const : double`

Jerarquía de Excepciones del Juego (std::exception) *Robustez y Manejo de Errores*

Clases Propias de Excepción:

- **ExcepcionJuegoBase** △ **std::exception**: Almacena `# mensajeDetalladoDelError : std::string` y sobreescribe `+ what() const noexcept : const char*`.
- **ExcepcionRecursoNoEncontrado** △ **ExcepcionJuegoBase**: Lanzada al fallar la apertura de sprites, fuentes o archivos de configuración.
- **ExcepcionMapaInvalido** △ **ExcepcionJuegoBase**: Lanzada cuando el archivo del mapa tiene dimensiones erróneas o zonas inaccesibles.
- **ExcepcionConfiguracionIncorrecta** △ **ExcepcionJuegoBase**: Lanzada si se introducen valores negativos o inconsistentes de dificultad.
- **ExcepcionEstadoInvalido** △ **ExcepcionJuegoBase**: Controla transiciones ilegales entre estados de juego (ej. pausar estando en Game Over).

3. Paquete 2: Modelos de Física Parametrizables (Requisito UdeA: ≥ 3 Físicas)

ModeloDeFisicaBase *Clase Abstracta de Cinemática*

Atributos Protegidos:

- `# puntoDeOrigenPivote : VectorDosDimensiones`
- `# tiempoAcumuladoEnSegundos : double = 0.0`
- `# posicionActualCalculada : VectorDosDimensiones`

Métodos Públicos y Virtuales:

- `+ virtual ~ModeloDeFisicaBase() = default`
- `+ virtual actualizarFisica(deltaTiempoEnSegundos: double) : void`
- `+ virtual calcularPosicionEnElInstante(tiempo: double) const : VectorDosDimensiones = 0`
- `+ obtenerPosicionActual() const : VectorDosDimensiones`
- `+ reiniciarModeloFisico() : void`
- `+ virtual haFinalizadoSuMovimiento() const : bool`

ModeloFisicoPenduloColgante △ ModeloDeFisicaBase *Física 1: Movimiento Oscilatorio*

Uso en el Juego: Lámpara colgante de un pasadizo colonial. Al balancearse proyecta luz y sombras en la pared que dificultan o revelan pistas ocultas.

- `- longitudDeLaCadena : double`
- `- anguloMaximoEnRadianes : double`
- `- frecuenciaDeOscilacion : double`
- `- factorDeAmortiguamiento : double`
- `- anguloActualEnRadianes : double`
- `+ ModeloFisicoPenduloColgante(origen, longitud, anguloMaximo, frecuencia, amortiguamiento)`
- `+ calcularPosicionEnElInstante(tiempo: double) const : VectorDosDimensiones override`
- `+ obtenerAnguloActualEnRadianes() const : double`
- `+ iluminarAreaAdyacente(puntoAExaminar: const VectorDosDimensiones&, radioDeLuz: double) const : bool`

ModeloFisicoMovimientoCircular △ ModeloDeFisicaBase *Física 2: Movimiento Periódico Circular*

Uso en el Juego: Patrulla orbital del vigilante que custodia el almacén de provisiones. Su radio y velocidad angular cambian con la dificultad.

- `- radioDeLaOrbitaCircular : double`
- `- velocidadAngularEnRadianes : double`
- `- anguloDeFaseInicial : double`
- `- giraEnSentidoHorario : bool`
- `+ ModeloFisicoMovimientoCircular(centroOrbita, radio, velocidadAngular, giraHorario)`
- `+ calcularPosicionEnElInstante(tiempo: double) const : VectorDosDimensiones override`
- `+ modificarRadioSegunDificultad(nuevoRadio: double) : void`
- `+ modificarVelocidadAngularSegunDificultad(nuevaVelocidadAngular: double) : void`

ModeloFisicoTiroParabolico △ ModeloDeFisicaBase *Física 3: Balística Parabólica*

Uso en el Juego: Balas de cañón disparadas desde las murallas británicas durante el Nivel 2. Representa la perturbación balística.

- `- velocidadInicialDelDisparo : double`
- `- anguloDeElevacionEnRadianes : double`
- `- aceleracionDeLaGravedad : double = 9.81`
- `- tiempoEstimadoDeVuelo : double`
- `- puntoFinalDeImpacto : VectorDosDimensiones`
- `- haImpactadoContraElSuelo : bool`
- `+ ModeloFisicoTiroParabolico(origenDisparo, velocidadInicial, anguloElevacion, gravedad)`
- `+ calcularPosicionEnElInstante(tiempo: double) const : VectorDosDimensiones override`
- `+ haFinalizadoSuMovimiento() const : bool override`
- `+ obtenerPuntoFinalDeImpacto() const : VectorDosDimensiones`

4. Paquete 3: Entorno, Mapa de Cartagena, Zonas y Rutas

MapaDeCartagena *Escenario y Cuadrícula de Navegación*

Atributos Privados:

- `- anchoEnCasillas : int`
- `- altoEnCasillas : int`
- `- tamanoDeCadaCasillaEnPixeles : int = 32`
- `- rejillaDeCasillas : std::vector<TipoDeCasilla>`
- `- listaDeZonasDelMapa : std::vector<ZonaDelEscenario>`
- `- listaDePuertasSecundarias : std::vector<PuertaBloqueable>`

Métodos Públicos:

- `+ MapaDeCartagena(ancho: int, alto: int, tamanoCasilla: int)`
- `+ esCasillaTransitable(coordenadaX: int, coordenadaY: int) const : bool`
- `+ esPosicionTransitableEnPixeles(posicion: const VectorDosDimensiones&) const : bool`
- `+ obtenerZonaEnPosicion(posicion: const VectorDosDimensiones&) const : int`
- `+ esZonaRestringida(identificadorDeZona: int) const : bool`
- `+ existeLineaDeVisionDirecta(origen: const VectorDosDimensiones&, destino: const VectorDosDimensiones&) const : bool`
- `+ cerrarPuertaSecundariaTemporalmente(idPuerta: int, segundosBloqueada: double) : void`
- `+ actualizarTemporizadoresDePuertas(deltaTiempo: double) : void`
- `+ desbloquearZonasExtendidasDelNivelDos() : void`

BuscadorDeRutasEstrella (Algoritmo A*) *Navegación Inteligente en Cuadrícula*

Propósito: Calcula la ruta más corta evitando muros, puertas cerradas y respetando permisos de zona.

- `- vectorDeCostesAcumulados : std::vector<double>`
- `- vectorDeNodosPadre : std::vector<int>`
- `- limiteMaximoDeNodosAExaminar : int = 500`
- `+ buscarCaminoOptimo(mapa: const MapaDeCartagena*, inicio: const VectorDosDimensiones&, destino: const VectorDosDimensiones&) : std::vector<VectorDosDimensiones>`
- `- calcularHeuristicaManhattan(x1: int, y1: int, x2: int, y2: int) const : double`

PuertaBloqueable & ZonaDelEscenario *Estructuras de Entorno*

- **PuertaBloqueable (struct):** `identificadorDePuerta : int`, `posicionEnCasilla : VectorDosDimensiones`, `estaCerradaActualmente : bool`, `segundosRestantesParaReabrir : double`, `esPuertaSecundaria : bool`.
- **ZonaDelEscenario (struct):** `identificadorDeZona : int`, `nombreDeLaZona : std::string`, `esDeAccesoRestringido : bool`, `grupoDeDesbloqueoNivelDos : int`.

5. Paquete 4: Jerarquía Polimórfica de Entidades del Juego

EntidadDelJuegoBase *Clase Abstracta Base de Escena*

Atributos Protegidos:

- `# identificadorUnicoDeEntidad : int`
- `# tipoDeEntidad : TipoDeEntidad`
- `# posicionActualEnMundo : VectorDosDimensiones`
- `# semianchoColision : double`
- `# semialtoColision : double`
- `# estaActivaEnEscena : bool = true`

Métodos Públicos y Virtuales:

- `+ virtual ~EntidadDelJuegoBase() = default`
- `+ virtual actualizarEntidad(deltaTiempo: double, escenario: MundoDeCartagena*) : void = 0`
- `+ verificarColisionCon(otraEntidad: const EntidadDelJuegoBase*) const : bool`
- `+ obtenerDistanciaHacia(otraEntidad: const EntidadDelJuegoBase*) const : double`
- `+ obtenerPosicionActual() const : VectorDosDimensiones`
- `+ establecerPosicion(nuevaPosicion: const VectorDosDimensiones&) : void`

PersonajeConMovimientoBase △ EntidadDelJuegoBase *Desplazamiento y Orientación*

Atributos Protegidos:

- `# velocidadBaseDeCaminata : double`
- `# multiplicadorDeVelocidadActual : double = 1.0`
- `# orientacionDeLaMirada : DireccionDeLaMirada`
- `# estadoDeLaAnimacion : EstadoDeAnimacion`
- `# vectorDeVelocidadActual : VectorDosDimensiones`
- `# zonaEnLaQueSeEncuentra : int`

Métodos Públicos:

- `+ virtual moverHaciaDireccion(direccion: const VectorDosDimensiones&, deltaTiempo: double, mapa: const MapaDeCartagena*) : bool`
- `+ detenerMovimientoCompletamente() : void`
- `+ virtual puedeCapturarAScooby() const : bool = 0`
- `+ aplicarRalentizacionTemporal(segundosDeDuracion: double, factorReduccion: double) : void`
- `+ obtenerVelocidadEfectiva() const : double`

PersonajeScooby △ PersonajeConMovimientoBase *Protagonista del Juego*

Atributos Privados:

- `- medidorDeEstamina : MedidorDeRecursoAcumulable` ♦ *(Aguanta 3 seg corriendo normalmente)*
- `- estaCorriendoRapido : bool = false`
- `- cantidadDeProvisionesQueTransporta : int = 0`
- `- capacidadMaximaDeProvisiones : int = 5`
- `- segundosRestantesDeSprintInfinito : double = 0.0` *(Comodín fruta: 12 seg sin cansarse)*
- `- segundosRestantesDeCamuflajePorDaphne : double = 0.0` *(Daphne lo oculta temporalmente)*

Métodos Públicos:

- `+ actualizarEntidad(deltaTiempo: double, escenario: MundoDeCartagena*) override`

- `+ procesarEntradaDelTeclado(direccionMovimiento: const VectorDosDimensiones&, teclaEspacioSprint: bool) : void`
- `+ activarComodinDeFrutaSprintInfinito(duracionEnSegundos: double = 12.0) : void`
- `+ ocultarTemporalmentePorDaphne(duracionEnSegundos: double) : void`
- `+ recolectarProvisionesParaLlevar(cantidadRecogida: int) : bool`
- `+ entregarProvisionesEnAlmacen() : int`
- `+ estaCompletamenteOculto() const : bool`
- `+ puedeCapturarAScooby() const override : bool { return false; }`

6. Paquete 4: Personajes Autónomos y Enemigos del Juego

PersonajeAutonomoConIA △ PersonajeConMovimientoBase *Base de Agentes Inteligentes*

Atributos Protegidos:

- `# cerebroDeInteligencia : CerebroAgenteAutonomo* ♦`
- `# temporizadorParaProximoPensamiento : double = 0.0`
- `# intervaloEntrePensamientosEnSegundos : double = 0.15`
- `# rutaDeNavegacionActual : std::vector<VectorDosDimensiones>`
- `# indiceDelPuntoDeRutaActual : size_t = 0`

Métodos Públicos:

- `+ actualizarEntidad(deltaTiempo: double, escenario: MundoDeCartagena*) override`
- `+ asignarRutaDeDestino(nuevaRuta: const std::vector<VectorDosDimensiones>&) : void`
- `+ limpiarRutaActual() : void`
- `+ tieneRutaActiva() const : bool`
- `+ transferirCerebroAlFantasma() : CerebroAgenteAutonomo*`

SospechosoInvestigado △ PersonajeAutonomoConIA *Los 4 Sospechosos*

Propósito: Representa a Aníbal, Solarte, Daniel o Sofía. Tienen actitudes sospechosas pero no persiguen por todo el mapa.

- `- nombreDelSospechoso : NombreSospechoso` *(ANIBAL, SOLARTE, DANIEL, SOFIA)*
- `- contadorDeEncuentrosConOtrosSospechosos : int = 0` *(Si se topan 3 veces, se repelen)*
- `- tiempoDeRepulsionActiva : double = 0.0`
- `- seHaDesvanecidoAlActivarTrampa : bool = false`
- `+ canCapturarAScooby() const override : bool { return true; }`
- `+ desvanecerDelEscenario() : void`
- `+ registrarEncuentroConColega() : void`

VillanoEncubierto △ SospechosoInvestigado *Villano Principal Elegido*

Propósito: Uno de los 4 sospechosos elegido aleatoriamente. Es sutil para no delatarse; usa habilidades encubiertas.

- `- nivelDeSospechaAcumulada : double = 0.0` *(Entre 0.0 y 1.0)*
- `- temporizadorParaAccionEspecial : double = 0.0`
- `+ ejecutarAccionEspecialDeVillano(mundo: MundoDeCartagena*) : void`
- `+ ordenarCierreDePuertaSecundaria(idPuerta: int, mapa: MapaDeCartagena*) : void`
- `+ solicitarPatrullaASoldadoAliado(zonaAInvestigar: int, mundo: MundoDeCartagena*) : void`
- `+ elevarVigilanciaEnZonaCaliente(idZona: int) : void`

SoldadoAliadoDelVillano △ PersonajeAutonomoConIA *Soldados de la Época*

Regla del Planteamiento: NO pueden capturar a Scooby. Lo persiguen temporalmente, le aplican ralentización (3 % más lento) y avisan a los superiores.

- `- estaEnAlertaPorLlamado : bool = false`
- `- temporizadorDePersecucionRestante : double = 0.0`
- `- contadorDePermisosParaZonaRestringida : int = 0` *(Aprende la puerta y tiene hasta 3 patrullas)*

- `- idPuertaPorDondeVioEntrarAScooby : int = -1`
- `+ canCapturarAScooby() const override : bool { return false; }` *(Exigencia de planteamiento)*
- `+ aplicarRalentizacionAScooby(scooby: PersonajeScooby*) : bool`
- `+ alertarASuperiores(zonaDeAvistamiento: int, mundo: MundoDeCartagena*) : void`

FantasmaDisfrazado △ **PersonajeAutonomoConIA** *Aparición Final tras Activar Trampa*

Regla del Planteamiento: Se genera al activar la trampa. Hereda el cerebro y la memoria del villano encubierto.

- `- estaEnfurecido : bool = true`
- `+ canCapturarAScooby() const override : bool { return true; }`
- `+ FantasmaDisfrazado(posicionInicial: const VectorDosDimensiones&, cerebroHeredado: CerebroAgenteAutonomo*)`

7. Paquete 4: Amigos de Scooby y Entidades Especiales del Entorno

AmigoSecundarioBase △ PersonajeAutonomoConIA *Amigos de la Máquina del Misterio*

Propósito: Personajes no hostiles que interactúan con Scooby cuando se aproxima.

- `# rolDelAmigo : RolAmigoSecundario` *(VELMA, FRED, SHAGGY, DAPHNE)*
- `# contadorDeConversacionesMantenidas : int = 0`
- `+ virtual interactuarConScooby(scooby: PersonajeScooby*, mundo: MundoDeCartagena*) : void = 0`
- `+ canCapturarAScooby() const override : bool { return false; }`

Especialización de los 4 Amigos *Habilidades según el Planteamiento*

- **AmigoVelma △ AmigoSecundarioBase:** Brinda información actualizada sobre el caso y el progreso de pistas.
- **AmigoFred △ AmigoSecundarioBase:** Entrega provisiones a Scooby cada 60 segundos de forma periódica.
- **AmigoShaggy △ AmigoSecundarioBase:** Se desplaza cerca de las pistas ocultas y da pistas sutiles sobre su paradero.
- **AmigaDaphne △ AmigoSecundarioBase:** Permanece en el punto de inicio. Si Scooby es perseguido, lo camufla por unos segundos. **Al activar la trampa, desaparece de la escena**.

CentinelaVigilanteDelAlmacen △ PersonajeConMovimientoBase *Física 2 en Entidad*

Propósito: Guardia español que patrulla en órbita circular alrededor del almacén de suministros.

- `- modeloDeOrbitaCircular : ModeloFisicoMovimientoCircular` ♦
- `+ actualizarEntidad(deltaTiempo: double, escenario: MundoDeCartagena*) override`
- `+ canCapturarAScooby() const override : bool { return false; }`
- `+ detectarIntrusoEnAreaDeSuministros(posicionScooby: const VectorDosDimensiones&) : bool`

LamparaOscilanteEnPared & BalaDeCanonMuralla *Físicas 1 y 3 en Entidades*

- **LamparaOscilanteEnPared △ EntidadDelJuegoBase:** Contiene `ModeloFisicoPenduloColgante` ♦. Oscila proyectando luz; revela marcas ocultas en los muros de Cartagena.
- **BalaDeCanonMuralla △ EntidadDelJuegoBase:** Contiene `ModeloFisicoTiroParabolico` ♦. Disparada periódicamente desde las murallas en el Nivel 2.

Objetos Coleccionables y Trampa *Items e Interacciones del Escenario*

- **PistaOcultaDelCaso △ EntidadDelJuegoBase:** Pista que ayuda a descartar sospechosos o encontrar la trampa. Puede requerir que la lámpara oscilante la ilumine.
- **SuministroDeProvisiones △ EntidadDelJuegoBase:** Paquete de víveres que Scooby recoge y debe transportar al almacén.
- **ComodinDeTiempoExtra △ EntidadDelJuegoBase:** Aparece cada menos de 2 minutos; suma tiempo automáticamente al pasar cerca.
- **ComodinDeSprintFruta △ EntidadDelJuegoBase:** Otorga 12 segundos de sprint continuo sin agotar la estamina.
- **TrampaParaElVillano △ EntidadDelJuegoBase:** Estado oculto → armada → activada. **¡Alerta!: Si Scooby pasa sobre ella una vez armada, él mismo queda atrapado y pierde la partida.**
- **AlmacenCentralDeProvisiones △ EntidadDelJuegoBase:** Punto de entrega vigilado donde Scooby deposita las provisiones transportadas.

8. Paquete 5: Arquitectura del Agente Autónomo (Componente Investigativo UdeA)

El diseño respeta estrictamente la separación conceptual en las 4 fases cognitivas exigidas por el departamento de sistemas:

CerebroAgenteAutonomo *Orquestador del Ciclo Cognitivo*

Atributos Privados (Composición de Módulos ♦):

- `- moduloDePercepcion : ModuloPercepcionSensorial*` ♦
- `- moduloDeRazonamiento : ModuloRazonamientoBase*` ♦
- `- moduloDeAccion : ModuloAccionBase*` ♦
- `- moduloDeMemoria : ModuloMemoriaYAprendizajeBase*` ♦
- `- personajePropietario : PersonajeConMovimientoBase*` *(Referencia sin propiedad)*

Métodos Públicos del Ciclo:

- `+ CerebroAgenteAutonomo(percepcion, razonamiento, accion, memoria, propietario)`
- `+ ejecutarCicloCognitivoCompleto(deltaTiempo: double, mundo: MundoDeCartagena*) : void`
- `- ejecutarFasePercepcion(mundo: MundoDeCartagena*) : EstimuloSensorialPercibido`
- `- ejecutarFaseRazonamiento(estimulo: const EstimuloSensorialPercibido&) : DecisionDeComportamiento`
- `- ejecutarFaseAccion(decision: const DecisionDeComportamiento&, mundo: MundoDeCartagena*) : ResultadoDeLaAccion`
- `- ejecutarFaseAprendizaje(estimulo, decision, resultado) : void`

ModuloPercepcionSensorial *Fase 1: Percepción*

Propósito: Filtra el mundo en 3 etapas: distancia euclidiana, cono frontal de visión y línea de vista directa sin muros.

- `- distanciaMaximaDeVision : double`
- `- anguloFrontalDelConoDeVision : double`
- `- puedePercibirSonidosDePasos : bool`
- `- consultaTableroDeAlertas : bool`
- `+ percibirEntorno(agente: const PersonajeConMovimientoBase*, mundo: const MundoDeCartagena*) : EstimuloSensorialPercibido`
- `- evaluarLineaDeVisionDespejada(origen, destino, mapa) const : bool`

Jerarquía de Razonamiento △ ModuloRazonamientoBase *Fase 2: Razonamiento*

Clase Abstracta y Derivadas Especializadas:

- **ModuloRazonamientoBase (Abstracto):** Mantiene `# estadoMentalActual : EstadoDeComportamiento`. Método virtual puro `+ decidirSiguienteAccion(estimulo, memoria) : DecisionDeComportamiento = 0`.
- **RazonamientoSospechosoComun △:** Evalúa patrullaje normal, curiosidad al ver a Scooby en zona prohibida y retorno ordenado a su ruta.
- **RazonamientoVillanoEncubierto △ RazonamientoSospechosoComun:** Evalúa además el nivel de sospecha; si detecta a Scooby 3 veces, ordena cerrar compuertas secundarias o convocar soldados aliados.
- **RazonamientoSoldadoAliado △:** Responde prioritariamente a alertas colectivas, inicia persecuciones temporales y busca frenar a Scooby.
- **RazonamientoAmigosSecundarios △:** Elige el diálogo de ayuda según las pistas encontradas y las conversaciones previas.

Jerarquía de Acción △ ModuloAccionBase *Fase 3: Acción*

Clase Abstracta y Derivadas:

- **ModuloAccionBase (Abstracto):** Traduce decisiones a comandos de movimiento o animación.
- **AccionSospechoso △:** Movimiento por ruta, detención a observar y animación de advertencia sonora sin revelar roles.
- **AccionVillanoAvanzado △:** Ejecuta cierre temporal de compuertas, asigna guardias y aumenta patrullas.
- **AccionSoldadoAliado △:** Ejecuta persecución activa y lanza el efecto de ralentización sobre Scooby.
- **AccionAmigoSecundario △:** Abre ventana de diálogo, entrega provisiones o camufla al jugador.

9. Paquete 5: Memoria, Aprendizaje Acumulativo y Alertas Compartidas

Jerarquía de Memoria △ ModuloMemoriaYAprendizajeBase *Fase 4: Aprendizaje*

Clase Abstracta Base:

- `# ultimaPosicionConocidaDeScooby : VectorDosDimensiones`
- `# seTieneRegistroPrevioDeScooby : bool = false`
- `# tiempoTranscurridoDesdeUltimoAvistamiento : double = 0.0`
- `+ virtual ~ModuloMemoriaYAprendizajeBase() = default`
- `+ virtual registrarExperiencia(estimulo, decision, resultado) = 0`
- `+ virtual desvanecerMemoriaPorPasoDelTiempo(deltaTiempo: double) : void`

Especializaciones de Memoria y Aprendizaje *Evidencia del Acto de Aprendizaje*

- **MemoriaAprendizajeVillano △ ModuloMemoriaYAprendizajeBase:**
  - `- mapaDeFrecuenciaPorZonas : std::array<int, 32>` *(Contenedor STL de tamaño fijo)*
  - `- nivelDeAlertaGeneral : double`
  - `+ registrarAvistamientoEnZona(identificadorDeZona: int) : void`
  - **Mecanismo de Aprendizaje:** Cuenta cuántas veces ha visto a Scooby en cada zona. Si el contador supera 3 avistamientos, eleva la prioridad de patrulla en esa zona y decide cerrar puertas secundarias o mandar guardias.
- **MemoriaAprendizajeSoldadoAliado △ ModuloMemoriaYAprendizajeBase:**
  - `- permisosDeEntradaAZonaRestringida : std::array<int, 32>`
  - `- puertaDeEntradaMemorizada : int`
  - **Mecanismo de Aprendizaje:** Si vio a Scooby ingresar a una zona prohibida, aprende la compuerta y obtiene un cupo de hasta 3 patrullas en esa área prohibida antes de que vuelva a serle vedada.
- **MemoriaAprendizajeAmigos △ ModuloMemoriaYAprendizajeBase:**
  - `- pistasYaInformadasAScooby : std::vector<int>`
  - `- numeroDeVecesHablado : int`
  - **Mecanismo de Aprendizaje:** Recuerda qué información ya le dio a Scooby para no repetir diálogos inútiles y ofrecer datos nuevos al avanzar la investigación.

TableroDeAlertasCompartido *Comunicación Desacoplada entre Agentes*

Propósito: Permite que el villano y los sospechosos alerten a los soldados aliados sin conocer la posición exacta de Scooby, manteniendo el realismo de la IA.

- `- listaDeAlertasActivas : std::vector<AlertaDeVigilancia>`
- `- capacidadMaximaDeAlertas : size_t = 16`
- `+ publicarNuevaAlerta(alerta: const AlertaDeVigilancia&) : void`
- `+ actualizarTemporizadoresDeAlertas(deltaTiempo: double) : void`
- `+ reclamarAlertaParaResponder(posicionSoldado: const VectorDosDimensiones&) : AlertaDeVigilancia*`
- `+ limpiarTodasLasAlertas() : void`

10. Paquete 6: Gestión de Niveles, Sesión y Conexiones Qt

SesionPrincipalDelJuego : QObject *Coordinador Maestro de la Partida*

Atributos Privados:

- `- nivelActualmenteActivo : NivelDelJuegoBase* ♦`
- `- relojMaestroDelJuego : RelojDeAltaPrecision* ♦`
- `- progresoGlobalDeCampana : ProgresoDeLaCampana ♦`
- `- perfilDeDificultadActivo : PerfilDeDificultad`
- `- colaDeEventosPendientes : ColaDeEventosDelJuego ♦`

Métodos Públicos y Slots:

- `+ iniciarNuevaPartida(dificultad: NivelDeDificultad) : void`
- `+ pausarSimulacion() : void`
- `+ reanudarSimulacion() : void`
- `+ alRecibirPasoDelReloj(deltaTiempo: double) : void` *(Slot del tick principal)*
- `+ procesarEntradaUsuario(entrada: const EstadoDeEntradaTeclado&) : void`

Señales Qt (`signals`):

- `+ senalTiempoRestanteActualizado(segundosRestantes: double)`
- `+ senalProvisionesEnAlmacenActualizadas(cantidadActual: int, metaRequerida: int)`
- `+ senalEstaminaDeScoobyActualizada(porcentajeEstamina: double)`
- `+ senalPistaDescubierta(textoDeLaPista: QString)`
- `+ senalTrampaArmadaEnElMapa(posicion: VectorDosDimensiones)`
- `+ senalScoobyHaSidoCapturado()`
- `+ senalCasoResueltoConExito(nombreDelCulpable: QString)`

NivelDelJuegoBase *Clase Abstracta de Nivel*

Atributos Protegidos:

- `# mundoDelEscenario : MundoDeCartagena* ♦`
- `# medidorDeTiempoLimite : MedidorDeRecursoAcumulable ♦`
- `# pistasNecesariasParaDeducir : int`
- `# pistasDescubiertasHastaAhora : int = 0`
- `# faseActualDeLaMision : FaseDeLaMision`

Métodos Públicos y Virtuales:

- `+ virtual ~NivelDelJuegoBase() = default`
- `+ virtual construirMundoYNivel() = 0`
- `+ virtual actualizarLogicaDelNivel(deltaTiempo: double) : void`
- `+ virtual evaluarCondicionDeFinDePartida() : ResultadoDelNivel = 0`
- `+ obtenerMundo() : MundoDeCartagena*`

NivelUnoPrimerasPistas △ NivelDelJuegoBase *Nivel 1: Tutorial Sutil*

Características del Nivel 1: Mapa reducido, condiciones sencillas para comprender mecánicas, interacción guiada con amigos y péndulo de lámpara colgante. Concluye al armar la trampa inicial y descubrir que el fantasma es un farsante disfrazado.

- `+ construirMundoYNivel() override`
- `+ evaluarCondicionDeFinDePartida() override`

NivelDosElMisterioPrincipal △ NivelDelJuegoBase *Nivel 2: Misterio Completo*

Características del Nivel 2: Mapa colonial ampliado (compuertas y pasadizos desbloqueados), 4 sospechosos reales con selección aleatoria del villano, aparición de comodines (tiempo, fruta de sprint, suministros), balas de cañón (tiro parabólico) y transformación del villano en el Fantasma al activar la trampa.

- `- expedienteDelCaso : ArchivoDelCasoCriminal ♦`
- `- generadorDeCasosAleatorios : GeneradorDeMisterios`
- `- sistemaDeDisparoDeCanones : SistemaDeBalasDeMuralla ♦`
- `+ construirMundoYNivel() override`
- `+ evaluarCondicionDeFinDePartida() override`
- `+ activarFaseDeTrampaYSpawnDelFantasma() : void`

11. Paquete 6: Generación del Caso Criminal y las 4 Dificultades

PerfilDeDificultad *Abstracción de Dificultad Exigida por UdeA*

Cuatro Modos Oficiales: `FACIL`, `MEDIO`, `DIFICIL`, `LEYENDA`. No se limita a cambiar el tiempo: parametriza la física, la IA y los recursos.

- `- nivelSeleccionado : NivelDeDificultad`
- `- numeroDePistasExigidas : int`
- `- tiempoInicialEnSegundos : double`
- `- velocidadDeDrenadoDeProvisiones : double`
- `- cantidadDeSoldadosAliados : int`
- `- rangoDeVisionDeLosEnemigos : double`
- `- velocidadAngularDelCentinelaCircular : double` *(Física 2 se acelera)*
- `- frecuenciaDelPenduloDeLaLampara : double` *(Física 1 oscila más rápido)*
- `+ static obtenerPerfilPara(dificultad: NivelDeDificultad) : PerfilDeDificultad`

ArchivoDelCasoCriminal & GeneradorDeMisterios *Proceduralidad*

- **ArchivoDelCasoCriminal (struct):** Almacena de forma inmutable durante la partida quién es el culpable real (`NombreSospechoso`), la posición de la trampa oculta (`VectorDosDimensiones`) y las ubicaciones de las pistas (`std::vector<UbicacionDePista>`).
- **GeneradorDeMisterios:** Elige aleatoriamente al culpable entre los 4 sospechosos (Aníbal, Solarte, Daniel, Sofía) y distribuye pistas y trampa en posiciones válidas del mapa sin solaparse.

12. Resumen Técnico de Relaciones, Herencia y Memoria Dinámica

Árbol Completo de Herencia Propia de C++ (Sin Qt)

- **EntidadDelJuegoBase** (Clase Base Polimórfica Pura)
  - △ **PersonajeConMovimientoBase**
    - △ **PersonajeScooby** (Protagonista con estamina y víveres)
    - △ **CentinelaVigilanteDelAlmacen** (Patrulla orbital continua)
    - △ **PersonajeAutonomoConIA**
      - △ **SospechosoInvestigado** (Aníbal, Solarte, Daniel, Sofía)
      - △ **VillanoEncubierto** (Sospechoso seleccionado con habilidades)
      - △ **SoldadoAliadoDelVillano** (Inmovilizadores militares)
      - △ **FantasmaDisfrazado** (Villano transformado en fase final)
      - △ **AmigoSecundarioBase**
      - △ **AmigoVelma**, △ **AmigoFred**, △ **AmigoShaggy**, △ **AmigaDaphne**
  - △ **LamparaOscilanteEnPared** (Péndulo)
  - △ **BalaDeCanonMuralla** (Parabólica)
  - △ **TrampaParaElVillano**, **AlmacenCentralDeProvisiones**, **PistaOcultaDelCaso**, **SuministroDeProvisiones**, **ComodinDeTiempoExtra**, **ComodinDeSprintFruta**
- **ModeloDeFisicaBase** (Cinemática pura)
  - △ **ModeloFisicoPenduloColgante**, △ **ModeloFisicoMovimientoCircular**, △ **ModeloFisicoTiroParabolico**
- **Módulos de IA Cognitiva**
  - △ **ModuloRazonamientoBase** → `RazonamientoSospechoso`, `RazonamientoVillano`, `RazonamientoSoldado`, `RazonamientoAmigo`
  - △ **ModuloAccionBase** → `AccionSospechoso`, `AccionVillano`, `AccionSoldado`, `AccionAmigo`
  - △ **ModuloMemoriaYAprendizajeBase** → `MemoriaVillano`, `MemoriaSoldado`, `MemoriaAmigo`

Propiedad y Jerarquía de Liberación de Memoria Dinámica (⧫)

Para prevenir fugas de memoria (*memory leaks*) y garantizar la destrucción segura:

- `SesionPrincipalDelJuego` ⧫ es dueña de `NivelDelJuegoBase*`.
- `NivelDelJuegoBase` ⧫ es dueño de `MundoDeCartagena*`.
- `MundoDeCartagena` ⧫ es dueño de todas las entidades en un `std::vector<EntidadDelJuegoBase*>`; en su destructor ejecuta un bucle llamando a `delete entidad` sobre cada una.
- `PersonajeAutonomoConIA` ⧫ es dueño de su `CerebroAgenteAutonomo*`.
- `CerebroAgenteAutonomo` ⧫ es dueño de sus 4 submódulos (`Percepcion*`, `Razonamiento*`, `Accion*`, `Memoria*`).
- Al activarse la trampa, el método `transferirCerebroAlFantasma()` cede la posesión del cerebro del villano al `FantasmaDisfrazado` y coloca el puntero original en `nullptr`, garantizando que se destruya exactamente una vez.

13. Secuencia de Ejecución: Tick del Reloj y Ciclo de Vida

Flujo Continuo en Cada Cuadro de Simulación (Delta Tiempo)

1. **Emisión del Reloj Maestro:** `RelojDeAltaPrecision` emite periódicamente `tick(deltaTiempo)` con un tope máximo de $\Delta t = 0{,}05$ s para prevenir inestabilidad numérica.
2. **Recepción en Sesión:** `SesionPrincipalDelJuego::alRecibirPasoDelReloj(dt)` delega la actualización en `NivelActivo->actualizarLogicaDelNivel(dt)`.
3. **Actualización de Entidades y Físicas:**
   - `PersonajeScooby`: Actualiza posición según teclado, disminuye estamina si sprinta y reduce temporizadores de comodines.
   - `LamparaOscilanteEnPared`: Ejecuta la ecuación del péndulo e ilumina sectores del muro.
   - `CentinelaVigilanteDelAlmacen`: Ejecuta la órbita circular alrededor del depósito.
   - `BalaDeCanonMuralla`: Avanza en trayectoria parabólica balística hasta tocar suelo.
4. **Ciclo Cognitivo de los Agentes Autónomos:**
   - Cada agente consulta si su temporizador ha expirado ($\Delta t_{pensar} \approx 0{,}15$ s).
   - Ejecuta en secuencia: **Percepción** → **Razonamiento** → **Acción** → **Aprendizaje**.
5. **Detección de Contactos y Colisiones:**
   - Si Scooby toca una pista → se registra en el expediente y suma tiempo.
   - Si Scooby entra al almacén con provisiones → las entrega y llena la barra del depósito.
   - Si Scooby pisa la trampa armada → Scooby queda atrapado (Derrota).
   - Si el Fantasma pisa la trampa armada → el Fantasma es atrapado (Victoria).
   - Si un sospechoso o el fantasma tocan a Scooby → Scooby es capturado (Derrota).
   - Si un soldado aliado toca a Scooby → aplica lentitud del 3 % y convoca alerta.
6. **Notificación a la GUI:** La sesión emite señales Qt hacia el HUD y las vistas gráficas.